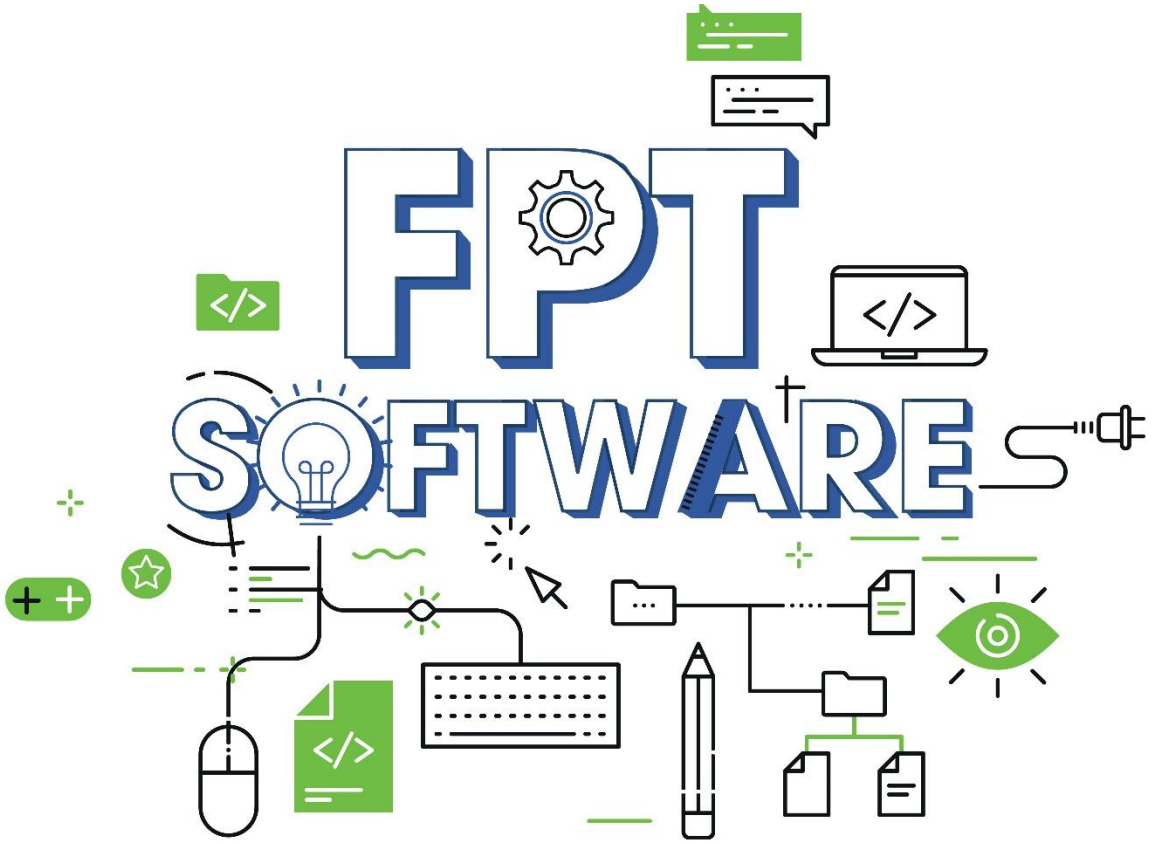
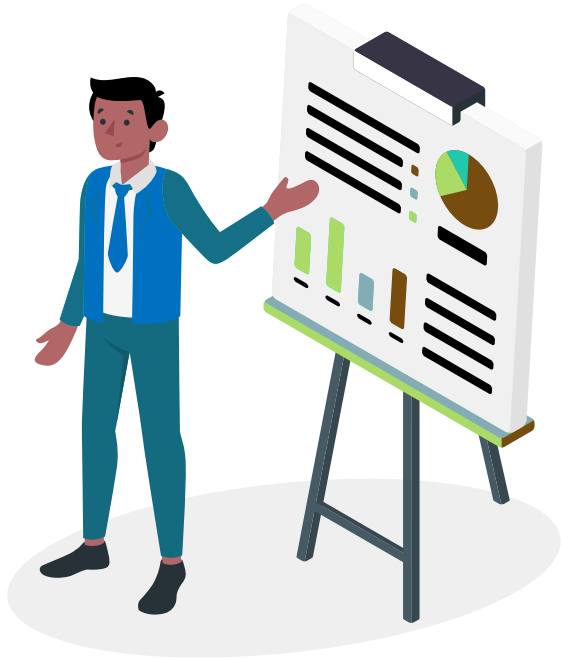
Basic Android

Animation



Agenda

1. Annimation Overview
2. View Animation
3. Property Animation
4. Drawable Animation



Lesson Objectives

01 Have basic understanding about Android's powerful animation APIs

02 Have basic understanding about the way to implement animation effects in an Android application

03 Have basic ability to create apps with flexible animation

Annimation Overview

Animation Overview

- The Android framework provides two animation systems: property animation (introduced in Android 3.0) and view animation.
- In addition to these two systems, you can utilize Drawable animation, which allows you to load drawable resources and display them one frame after another.
- Property Animation
  - ✓ Lets you animate properties of any object, including ones that are not rendered to the screen.
- View Animation
  - ✓ Can only be used for Views. It is relatively easy to setup and offers enough capabilities to meet many application's needs.
- Drawable Animation
  - ✓ Involves displaying Drawable resources one after another, like a roll of film.

View Animation

View Animation

- You can use the view animation system to perform **tweened animation** on Views.
- Tween animation calculates the animation with information such as the **start point**, **end point**, **size**, **rotation**, and other common aspects of an animation
- Can perform a series of simple **transformations** (**position**, **size**, **rotation**, and **transparency**) on the contents of a View object.
- So, if you have a TextView object, you can:
  - ✓ Move,
  - ✓ Rotate,
  - ✓ Grow,
  - ✓ Or Shrink the text.
- The animation package (**android.view.animation**) provides all the classes used in a tween animation.

View Animation

- Transformations can be **sequential** or **simultaneous**.
- Each transformation takes:
  - ✓ a set of parameters specific for that transformation (starting size and ending size for size change, starting angle and ending angle for rotation, and so on).
  - ✓ and also a set of common parameters (for instance, start time and duration).
- To make several transformations happen **simultaneously**, give them the same start time; to make them **sequential**, calculate the start time plus the duration of the preceding transformation
- A sequence of animation instructions defines the tween animation, defined by either **XML** or **Android code**.
- Defining by **XML** is recommended because it's **more readable**, **reusable**, and **swappable** than hard-coding the animation.

```
<set xmlns:android="http://schemas.android.com/apk/res/android"
    android:interpolator="@android:anim/linear_interpolator">
    <rotate
        android:duration="1000"
        android:fromDegrees="0"
        android:pivotX="50%"
        android:pivotY="50%"
        android:repeatCount="infinite"
        android:startOffset="0"
        android:toDegrees="360" />
</set>
```

View Animation

- The animation XML file belongs in the **res/anim/** directory of your Android project.
  - ✓ The file must have a single root element: this will be either a single **<alpha>**, **<scale>**, **<translate>**, **<rotate>**, interpolator element, or **<set>** element that holds groups of these elements (which may include another **<set>**).
  - ✓ By **default**, all animation instructions are applied **simultaneously**. To make them occur **sequentially**, you must specify the **startOffset** attribute
- Screen coordinates:
  - ✓ are (0,0) at the upper left hand corner.
  - ✓ and increase as you go down and to the right.
- Some values, such as pivotX, can be specified relative to the object itself or relative to the parent.
  - ✓ For example: "50" for 50% relative to the parent
  - ✓ or "50%" for 50% relative to itself
- You can determine how a transformation is applied over time by assigning an Interpolator.
- Android includes several **Interpolator** subclasses that specify various speed curves.
  - ✓ For instance: **AccelerateInterpolator** tells a transformation to start slow and speed up.

View Animation

- The following code will reference XML file saved in the **res/anim/** directory and apply it to an ImageView object from the layout.

```
ImageView myView = findViewById(R.id.animatorImg);
Animation animation = AnimationUtils.loadAnimation( context: this, R.anim.rotate_clockwise);
myView.startAnimation(animation);
```

- As an alternative to **startAnimation()**, you can define a starting time for the animation with **Animation.setStartTime()**, then assign the animation to the View with **View.setAnimation()**
- To define tween animation in Android's code, these common subclasses are used:
  - ✓ AlphaAnimation - <alpha>
  - ✓ ScaleAnimation - <scale>
  - ✓ TranslateAnimation - <translate>
  - ✓ RotateAnimation - <rotate>
  - ✓ AnimationSet - <set>
- For example:

```
ScaleAnimation animation = new ScaleAnimation(fromXscale, toXscale, fromYscale, toYscale,
        Animation.RELATIVE_TO_SELF, (float) 0.5, Animation.RELATIVE_TO_SELF, (float) 0.5);
```

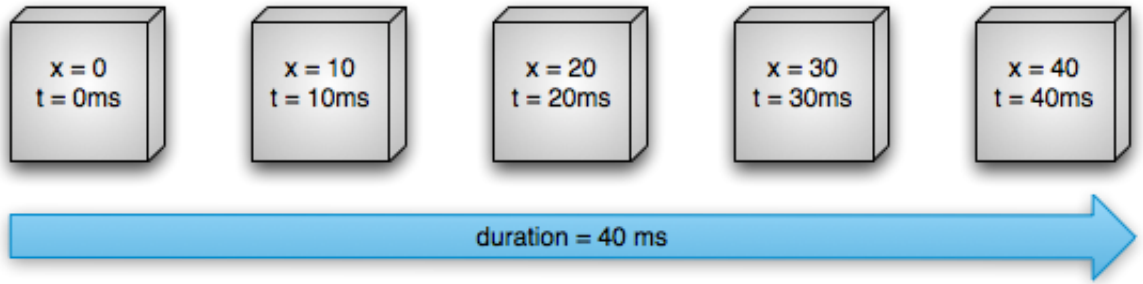
Property Animation

Property Animation

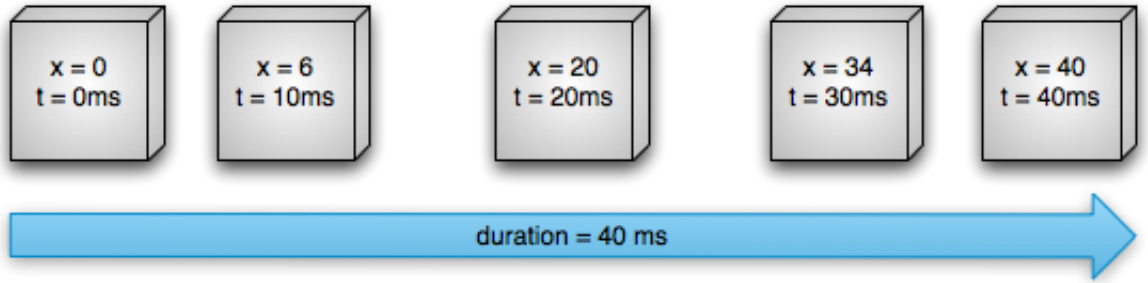
- The property animation system is a robust framework that allows you to animate almost anything.
- A property animation changes a property's (a field in an object) value over a specified length of time.
- To animate something, you specify the object property that you want to animate, such as:
  - an **object's position** on the screen.
  - **how long** you want to animate it for.
  - and **what values** you want to animate between.
- The property animation system lets you define the following characteristics of an animation:
  - **Duration**: You can specify the duration of an animation.
  - **Time interpolation**: You can specify how the values for the property are calculated as a function of the animation's current elapsed time.
  - **Repeat count and behavior**: You can specify whether or not to have an animation repeat when it reaches the end of a duration and how many times to repeat the animation.
  - **Animator sets**: You can group animations into logical sets that play together or sequentially or after specified delays.
  - **Frame refresh delay**: You can specify how often to refresh frames of your animation. The default is set to refresh every 10 ms.

How Property Animation Works

- Linear animation:

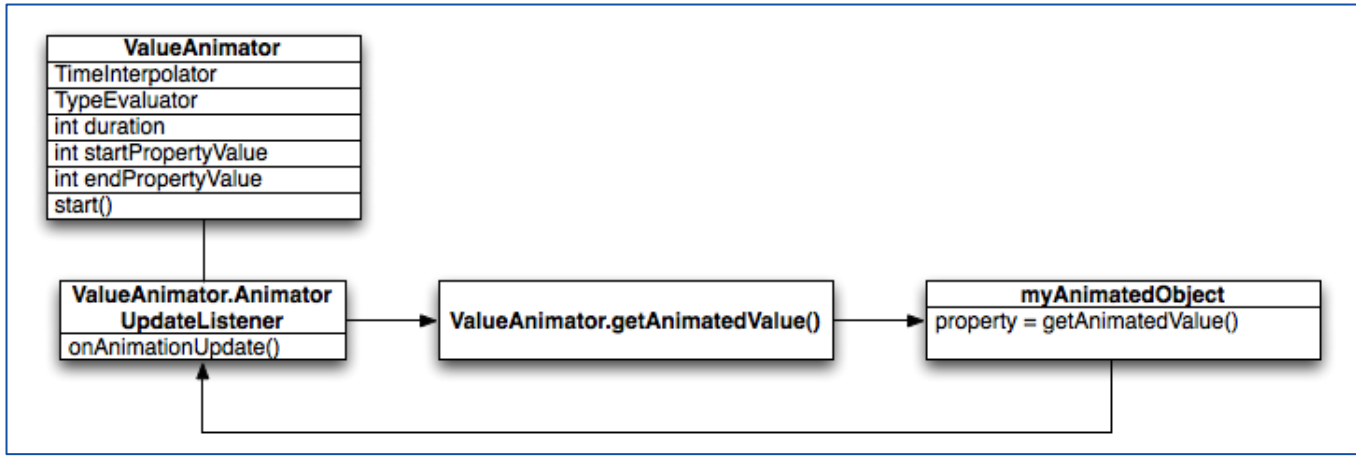


- Non-linear animation:



How Property Animation Works

- How animations are calculated:
- The **ValueAnimator** object keeps track of your animation's timing, such as how long the animation has been running, and the current value of the property that it is animating.
- The **ValueAnimator** encapsulates a **TimeInterpolator**, which defines animation interpolation, and a **TypeEvaluator**, which defines how to calculate values for the property being animated.
- When you call **start()** the animation begins.
- During the whole animation, the **ValueAnimator** calculates an elapsed fraction between 0 and 1.
- When it is done calculating, it calls the **TimeInterpolator** that is currently set, to calculate an interpolated fraction.



- When the interpolated fraction is calculated, **ValueAnimator** calls the appropriate **TypeEvaluator**:
  - ✓ to calculate the value of the property that you are animating
  - ✓ based on the interpolated fraction, the starting value, and the ending value of the animation.

How Property Anim Differs from View Anim

❖The view animation system:

- Provides the capability to only animate View objects.
- Is also constrained in the fact that it only exposes a few aspects of a View object to animate.
- Only modified where the View was drawn, and not the actual View itself.
- Takes less time to setup and requires less code to write.

❖With the property animation system:

- You can animate any property of any object (Views and non-Views).
- These constraints are completely removed. These constraints are completely removed.
- The object itself is actually modified. The object itself is actually modified.
- The property animation is more robust in the way it carries out animation with many types of properties (color, position, size) and any aspects of the animation (interpolation, synchronization).

API Overview

- You can find most of the property animation system's APIs in android.animation.
- Main components of the property animation system:
  - ✓ The **Animator** class provides the basic structure for creating animations.
  - ✓ **Evaluators** tell the property animation system how to calculate values for a given property.
  - ✓ A **time interpolator** defines how specific values in an animation are calculated as a function of time.

Animating with ValueAnimator

- This class lets you animate values of some type for the duration of an animation by specifying a set of int, float, or color values to animate through.
- You obtain a ValueAnimator by calling one of its factory methods: ofInt(), ofFloat(), or ofObject().

```
ValueAnimator animation = ValueAnimator.ofFloat( ...values: 0f, 1f);
animation.setDuration(1000);
animation.start();
```

- In this code, the ValueAnimator starts calculating the values of the animation, between 0 and 1, for a duration of 1000 ms, when the start() method runs.

Animating with ObjectAnimator

- This is a subclass of the **ValueAnimator** and combines the timing engine and value computation of **ValueAnimator** with the ability to animate a named property of a target object.
- Instantiating an **ObjectAnimator** is similar to a ValueAnimator, but you also specify the object and the name of that object's property (as a String) along with the values to animate between:

```
ObjectAnimator anim = ObjectAnimator.ofFloat(foo, propertyName: "alpha", ...values: 0f, 1f);
anim.setDuration(1000);
anim.start();
```

Choreographing Multiple Anim with AnimatorSet

- The Android system lets you bundle animations together into an **AnimatorSet**.
- So that you can specify whether to start animations simultaneously, sequentially, or after a specified delay.

```
AnimatorSet animatorSet= new AnimatorSet();
animatorSet.play(anim1).before(anim2);
animatorSet.play(anim1).with(anim3);
animatorSet.play(anim1).after(anim4);
animatorSet.start();
```

Animation Listeners

- You can listen for important events during an animation's duration with the listeners described below.
- **Animator.AnimatorListener**
  - ✓ **onAnimationStart()** - Called when the animation starts.
  - ✓ **onAnimationEnd()** - Called when the animation ends.
  - ✓ **onAnimationRepeat()** - Called when the animation repeats itself.
  - ✓ **onAnimationCancel()** - Called when the animation is canceled. A cancelled animation also calls **onAnimationEnd()**, regardless of how they were ended.
- **ValueAnimator.AnimatorUpdateListener**
  - ✓ **onAnimationUpdate()** - called on every frame of the animation.
    - Listen to this event to use the calculated values generated by ValueAnimator during an animation.
    - To use the value, query the ValueAnimator object passed into the event to get the current animated value with the getAnimatedValue() method. Implementing this listener is required if you use ValueAnimator.
    - Depending on what property or object you are animating, you might need to call **invalidate()** on a View to force that area of the screen to redraw itself with the new animated values.
- You can extend the **AnimatorListenerAdapter** class instead of implementing the **Animator.AnimatorListener** interface, if you do not want to implement all of the methods of the **Animator.AnimatorListener** interface.
- The **AnimatorListenerAdapter** class provides empty implementations of the methods that you can choose to override.

Animating Layout Changes to ViewGroups

- You can animate layout changes within a **ViewGroup** with the **LayoutTransition** class.
  - ✓ Views inside a **ViewGroup** can go through an appearing animation when:
    - you add them to a **ViewGroup**
    - or when you call a View's **setVisibility()** method with **VISIBLE**
    - when you **add** or **remove** Views.
  - ✓ Or it can go through an disappearing animation when:
    - you remove them from a **ViewGroup**
    - Or when you call a View's **setVisibility()** method with **INVISIBLE** or **GONE**.
- You can define the following animations in a **LayoutTransition** object by calling **setAnimator()** and passing in an **Animator** object with one of the following **LayoutTransition** constants:
  - ✓ **APPEARING** - A flag indicating the animation that runs on items that are appearing in the container.
  - ✓ **CHANGE_APPEARING** - A flag indicating the animation that runs on items that are changing due to a new item appearing in the container.
  - ✓ **DISAPPEARING** - A flag indicating the animation that runs on items that are disappearing from the container.
  - ✓ **CHANGE_DISAPPEARING** - A flag indicating the animation that runs on items that are changing due to an item disappearing from the container.

Using TypeEvaluator & Interpolators

- If you want to animate a type that is unknown to the Android system, you can create your own evaluator by implementing the **TypeEvaluator** interface:
  - ✓ The types that are known by the Android system are **int**, **float**, or a **color**, which are supported by the **IntEvaluator**, **FloatEvaluator**, and **ArgbEvaluator** type evaluators.
  - ✓ There is only one method to implement in the **TypeEvaluator** interface, the **evaluate()** method.

```
public class FloatEvaluator implements TypeEvaluator {
    @Override
    public Object evaluate(float fraction, Object startValue, Object endValue) {
        float startFloat = ((Number) startValue).floatValue();
        return startFloat + fraction * (((Number) endValue).floatValue() - startFloat);
    }
}
```

- An interpolator define how specific values in an animation are calculated as a function of time.
  - ✓ AccelerateDecelerateInterpolator

```
@Override
public float getInterpolation(float input) {
    return (float)(Math.cos((input + 1) * Math.PI) / 2.0f) + 0.5f;
}
```

  - ✓ LinearInterpolator

```
@Override
public float getInterpolation(float input) {
    return input;
}
```

- You can implement the **TimeInterpolator** interface and create your own.

Specify keyframes

- A **Keyframe** object consists of a time/value pair that lets you define a specific state at a specific time of an animation
- To instantiate a Keyframe object, you must use one of the factory methods, ofInt(), ofFloat(), or ofObject() to obtain the appropriate type of Keyframe. You then call the ofKeyframe() factory method to obtain a PropertyValuesHolder object. Once you have the object, you can obtain an animator by passing in the PropertyValuesHolder object and the object to animate

```
Keyframe kf0 = Keyframe.ofFloat( fraction: 0f,  value: 0f);
Keyframe kf1 = Keyframe.ofFloat( fraction: .5f,  value: 360f);
Keyframe kf2 = Keyframe.ofFloat( fraction: 1f,  value: 0f);
PropertyValuesHolder pvhRotation = PropertyValuesHolder.ofKeyframe( propertyName: "rotation", kf0, kf1, kf2);
ObjectAnimator rotationAnim = ObjectAnimator.ofPropertyValuesHolder(target, pvhRotation);
rotationAnim.setDuration(5000);
```

Animating Views

- The new properties in the **View** class that facilitate property animations are:
  - ✓ **translationX** and **translationY**: These properties control where the View is located as a delta from its left and top coordinates which are set by its layout container.
  - ✓ **rotation**, **rotationX**, and **rotationY**: These properties control the rotation in 2D (rotation property) and 3D around the pivot point.
  - ✓ **scaleX** and **scaleY**: These properties control the 2D scaling of a View around its pivot point.
  - ✓ **pivotX** and **pivotY**: These properties control the location of the pivot point, around which the rotation and scaling transforms occur. By default, the pivot point is located at the center of the object.
  - ✓ **x** and **y**: These are simple utility properties to describe the final location of the View in its container, as a sum of the left and top values and translationX and translationY values.
  - ✓ **alpha**: Represents the alpha transparency on the View. This value is 1 (opaque) by default, with a value of 0 representing full transparency (not visible).
- **For example:** ObjectAnimator.ofFloat(myView, **"rotation"**, 0f, 360f);

Animating Views

- The property animation system allow streamlined animation of View objects and offers a few advantages over the view animation system.
- The property animation system can animate **Views** on the screen by changing the actual properties in the **View** objects.
- In addition, **Views** also automatically call the **invalidate()** method to refresh the screen whenever its properties are changed.
- The property animation system allow streamlined animation of View objects and offers a few advantages over the view animation system.
- The property animation system can animate **Views** on the screen by changing the actual properties in the **View** objects.
- In addition, **Views** also automatically call the **invalidate()** method to refresh the screen whenever its properties are changed.

Animating with ViewPropertyAnimator

- The **ViewPropertyAnimator** provides a simple way to animate several properties of a **View** in parallel, using a single underlying **Animator** object:
- Multiple ObjectAnimator objects

```
ObjectAnimator animX = ObjectAnimator.ofFloat(myView, propertyName: "x", ...values: 50f);
ObjectAnimator animY = ObjectAnimator.ofFloat(myView, propertyName: "y", ...values: 100f);
AnimatorSet animSetXY = new AnimatorSet();
animSetXY.playTogether(animX, animY);
animSetXY.start();
```

- One ObjectAnimator

```
PropertyValuesHolder pvhX = PropertyValuesHolder.ofFloat( propertyName: "x", ...values: 50f);
PropertyValuesHolder pvhY = PropertyValuesHolder.ofFloat( propertyName: "y", ...values: 100f);
ObjectAnimator.ofPropertyValuesHolder(myView, pvhX, pvhY).start();
```

- ViewPropertyAnimator

```
myView.animate().x( value: 50f).y( value: 100f);
```

Declaring Animations in XML

- The property animation system lets you declare property animations with XML instead of doing it programmatically.
- The following property animation classes have XML declaration support with the following XML tags:
  - ✓ **ValueAnimator - <animator>**
  - ✓ **ObjectAnimator - <objectAnimator>**
  - ✓ **AnimatorSet - <set>**
- For example:
  - ✓ Create xml object animator

```
<set xmlns:android="http://schemas.android.com/apk/res/android">
    <objectAnimator
        android:propertyName="alpha"
        android:duration="500"
        android:valueTo="1">
    </objectAnimator>
</set>
```

  - ✓ Load object animator:

```
AnimatorSet set = (AnimatorSet) AnimatorInflater.loadAnimator( context: this,  R.animator.object_animator);
set.setTarget(myView);
set.start();
```

Drawable Animation

Drawable Animation

- Drawable animation lets you load a series of Drawable resources one after another to create an animation.
- This is a traditional animation in the sense that it is created with a sequence of different images, played in order, like a roll of film.
- The **AnimationDrawable** class is the basis for Drawable animations.
- While you can define the frames of an animation in your code, using the AnimationDrawable class API.
- The XML file for this kind of animation belongs in the **res/drawable/** directory of your Android project.
- The XML file consists of an **<animation-list>** element as the root node and a series of child **<item>** nodes that each define a frame: a drawable resource for the frame and the frame duration.

Drawable Animation

- Example XML file for a Drawable animation.

```xml
<?xml version="1.0" encoding="utf-8"?>
<animation-list xmlns:android="http://schemas.android.com/apk/res/android"
    android:oneshot="true">
    <item
        android:drawable="@drawable/HDI_Booting_15sec_WarmV008"
        android:duration="200" />
    <item
        android:drawable="@drawable/HDI_Booting_15sec_WarmV009"
        android:duration="200" />
    <item
        android:drawable="@drawable/HDI_Booting_15sec_WarmV010"
        android:duration="200" />
</animation-list>
```

- ✓ This animation runs for just three frames.
- ✓ By setting the android:oneshot attribute of the list to ***true***, it will cycle just once then stop and hold on the last frame. If it is set ***false*** then the animation will loop.
- ✓ This XML can be added as the background image to a View.

Drawable Animation

- Example code for a Drawable animation.

```
ImageView myImage = (ImageView) findViewById(R.id.my_image);
myImage.setBackgroundResource(R.drawable.animation);
AnimationDrawable myAnimation = (AnimationDrawable) myImage.getBackground();
myAnimation.start();
```

  - ➢ It's important to note that the **start**() method called on the AnimationDrawable cannot be called during the **onCreate**() method of your Activity, because the AnimationDrawable is not yet fully attached to the window.
  - If you want to play the animation immediately, without requiring interaction, then you might want to call it from the **onWindowFocusChanged**() method in your Activity, which will get called when Android brings your window into focus.

THANK YOU!